

Applied Machine Learning Lab

B.Tech Semester 4

Experiment 04

Telecom Customer Churn Prediction using Machine Learning Models

Date: 4th feb 2026

SAP ID: 590016154

Name: Vidisha

AIM

To build and compare multiple machine learning models to predict telecom customer churn and identify key factors influencing customer retention.

THEORY

Customer churn prediction is a classification problem. Models like Logistic Regression, KNN, SVM, Decision Tree and Random Forest are used.

Evaluation metrics include Accuracy, Precision, Recall, F1-score and AUC-ROC.

ROC curves, confusion matrix and feature importance help in understanding model performance.

Commands/Code Used

```
# Import libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.svm import SVC
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,  
roc_auc_score, confusion_matrix, roc_curve
```

```
# Load dataset
```

```
df = pd.read_csv("Telco-Customer-Churn.csv")
```

```
# Preprocessing
```

```
df = df.drop_duplicates()
```

```
# Encode categorical data
```

```
le = LabelEncoder()

for col in df.select_dtypes(include='object'):

    df[col] = le.fit_transform(df[col])


# Feature selection

X = df.drop("Churn", axis=1)

y = df["Churn"]


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)


# Models

models = {

    "Logistic Regression": LogisticRegression(),

    "KNN": KNeighborsClassifier(),

    "SVM": SVC(probability=True),

    "Decision Tree": DecisionTreeClassifier(),

    "Random Forest": RandomForestClassifier()

}


# Store results
```

```
results = {}
```

```
# Train and evaluate
```

```
for name, model in models.items():
```

```
    model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)
```

```
    y_prob = model.predict_proba(X_test)[:, 1]
```

```
    results[name] = {
```

```
        "Accuracy": accuracy_score(y_test, y_pred),
```

```
        "Precision": precision_score(y_test, y_pred),
```

```
        "Recall": recall_score(y_test, y_pred),
```

```
        "F1 Score": f1_score(y_test, y_pred),
```

```
        "AUC": roc_auc_score(y_test, y_prob)
```

```
    }
```

```
    print(f"\n{name}")
```

```
    print(results[name])
```

```
# Convert results to DataFrame
```

```
results_df = pd.DataFrame(results).T
```

```
# Plot comparison
```

```
results_df.plot(kind='bar', figsize=(10,6))
```

```
plt.title("Model Performance Comparison")
```

```
plt.ylabel("Score")
plt.xticks(rotation=45)
plt.show()
```

```
# ROC Curves
```

```
plt.figure(figsize=(8,6))
```

```
for name, model in models.items():
    y_prob = model.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    plt.plot(fpr, tpr, label=name)
```

```
plt.plot([0,1], [0,1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.show()
```

```
# Confusion Matrix (for best model - Random Forest)
```

```
best_model = models["Random Forest"]
y_pred = best_model.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
```

```
plt.title("Confusion Matrix - Random Forest")
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.show()
```

```
# Feature Importance (Random Forest)
```

```
importances = best_model.feature_importances_
```

```
features = X.columns
```

```
importance_df = pd.DataFrame({"Feature": features, "Importance": importances})
```

```
importance_df = importance_df.sort_values(by="Importance")
```

```
plt.figure(figsize=(8,6))
```

```
plt.barh(importance_df["Feature"], importance_df["Importance"])
```

```
plt.title("Feature Importance")
```

```
plt.show()
```

RESULTS

Feature importance analyzed using Random Forest.

Model performance compared using evaluation metrics.

ROC curves plotted for all models.

Confusion matrices generated.

CONCLUSION

Multiple models were trained and evaluated. Random Forest performed well. The experiment showed importance of model comparison and evaluation.